# Week 2, Day 1: Spring IoC/DI, Bean Lifecycle and Binary Search

**Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: IoC, DI, bean lifecycle, scopes, circular dependencies | DSA: Binary search**

## Today's Schedule

- **0:00–0:45** Study IoC, DI types and bean definition styles (sections 1 to 3).
- **0:45–1:30** Study the bean lifecycle and scopes (sections 4 and 5).
- **1:30–1:50** Study circular dependencies (section 6), then try each example in a small Spring Boot project.
- **1:50–2:50** DSA: solve the 3 binary search problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **2:50–3:20** Answer the interview questions aloud (section 8) and add 10 flashcards.

## 1. IoC and the Spring Container

- **Inversion of Control (IoC):** instead of your code creating and wiring its dependencies with new, the container creates the objects (beans), injects their dependencies and manages their lifecycle.
- **Dependency Injection (DI)** is the technique Spring uses to apply IoC. Classes declare what they need, and Spring supplies it.
- **BeanFactory:** the basic container. It creates beans lazily, when first requested.
- **ApplicationContext:** the full-featured container used in practice. It extends BeanFactory with event publishing, internationalization, environment and profile support, and AOP integration. By default it creates **singleton beans eagerly at startup**, so wiring problems show up immediately instead of at the first request.
- **Bean definition:** the recipe the container uses to create a bean (class, scope, dependencies, init and destroy methods). Definitions come from component scanning, @Bean methods, or XML in older projects.
- **Why it matters:** loose coupling, easy replacement of implementations, simple testing with mocks, and centralized lifecycle management.

## 2. Types of Dependency Injection

### Constructor injection (recommended)

- Dependencies are final, required and visible. The object can never exist in a half-built state.
- Easy to unit test without Spring, by passing mocks to the constructor.
- Since Spring 4.3, a class with a single constructor does not need @Autowired.
- Makes circular dependencies fail fast at startup.

**Setter injection**

- Good for optional dependencies that have a sensible default, or that may be reconfigured.
- The object can be used before the setter is called, which is a risk.

**Field injection (avoid)**

- Short to write, but hides dependencies, cannot make fields final, and cannot be built without reflection or a Spring test context.

```
@Service
public class OrderService {
    private final OrderRepository repository;
    private final PaymentGateway gateway;

    // single constructor: @Autowired is optional
    public OrderService(OrderRepository repository, PaymentGateway gateway) {
        this.repository = repository;
        this.gateway = gateway;
    }
}
```

**Resolving ambiguity when there are several candidates**

- Spring matches by **type** first. If more than one bean fits, it tries the **name** (parameter or field name), then qualifiers.
- **@Primary:** marks the default candidate.
- **@Qualifier("name"):** picks a specific bean at the injection point.
- **Inject all implementations:** declare List<PaymentGateway> or Map<String, PaymentGateway>. This is a clean way to implement the Strategy pattern.
- **Optional dependencies:** use Optional<T>, @Autowired(required = false), or ObjectProvider<T>.
- **@Resource** (jakarta.annotation) injects by name first, while @Autowired injects by type first.

# 3. Defining Beans: @Component vs @Bean

- **Stereotype annotations:** @Component is the generic one. @Service and @Controller (or @RestController) add meaning. **@Repository** also enables translation of persistence exceptions into Spring's DataAccessException hierarchy.
- **@Component** goes on your own classes and is found by component scanning.
- **@Bean** goes on a method inside a @Configuration class and returns the object to register. Use it for third-party classes you cannot annotate (for example an ObjectMapper or a RestTemplate), or when creation needs custom logic.
- **@SpringBootApplication** combines @Configuration, @EnableAutoConfiguration and @ComponentScan. Scanning starts at the package of the main class and goes down, so classes outside it are not found.
- **@Configuration vs lite mode:** a @Configuration class is wrapped in a CGLIB proxy, so calling one @Bean method from another returns the same singleton instead of creating a new object. @Bean methods inside a plain @Component are in "lite mode" and do not get that guarantee.

```
@Configuration
public class AppConfig {
    @Bean
    public RestClient paymentClient(@Value("${payment.url}") String baseUrl)
{
        return RestClient.builder().baseUrl(baseUrl).build();
    }

    @Bean
    public PaymentGateway paymentGateway(RestClient paymentClient) {
        return new HttpPaymentGateway(paymentClient);
    }
}
```

# 4. Bean Lifecycle

### Order of events for a singleton bean

1. **Bean definitions loaded.** BeanFactoryPostProcessors can modify definitions before any bean is created (this is how property placeholders are resolved).
2. **Instantiation:** the constructor runs.
3. **Dependency injection:** fields and setters are populated.
4. **Aware callbacks:** BeanNameAware, BeanFactoryAware, ApplicationContextAware, if implemented.

5. **BeanPostProcessor, before initialization.**
6. **Initialization callbacks, in this order:** @PostConstruct, then InitializingBean.afterPropertiesSet(), then a custom init-method.
7. **BeanPostProcessor, after initialization.** This is where Spring wraps beans in **AOP proxies** (for @Transactional, @Async, @Cacheable and so on).
8. **Bean is ready for use.**
9. **On shutdown:** @PreDestroy, then DisposableBean.destroy(), then a custom destroy-method.

**Points to remember**

- In Spring Boot 3, @PostConstruct and @PreDestroy come from the **jakarta.annotation** package (they were javax before).
- Do not rely on injected dependencies inside the constructor of a field-injected bean, because they are not set yet. Use @PostConstruct, or better, constructor injection.
- The container does **not** manage the destruction of prototype beans. It creates them and hands them over.
- BeanPostProcessors are the extension point behind many Spring features, including @Autowired processing and proxy creation.

```
@Component
public class CacheWarmer {
    private final ProductRepository repository;
    public CacheWarmer(ProductRepository repository) { this.repository =
repository; }

    @PostConstruct
    void warmUp() { repository.findTop100(); }

    @PreDestroy
    void cleanUp() { /* release resources */ }
}
```

## 5. Bean Scopes

- **singleton (default):** one instance per container (not per JVM). Because many threads share it, a singleton must be **stateless or thread-safe**. Do not keep per-request data in fields.
- **prototype:** a new instance every time it is requested.
- **request, session, application, websocket:** web-aware scopes, valid only in a web application.

### The prototype-inside-singleton trap

A singleton is built once, so a prototype injected into it is also created only once, and the "new instance each time" behavior is lost. Fixes:

- Inject **ObjectProvider<T>** and call getObject() each time you need a fresh instance.
- Use a **scoped proxy** (@Scope with proxyMode).
- Use @Lookup method injection, which is rarer.

```
@Component
class ReportService {
    private final ObjectProvider<ReportBuilder> builders;
    ReportService(ObjectProvider<ReportBuilder> builders) { this.builders =
builders; }

    Report create() {
        ReportBuilder builder = builders.getObject();   // fresh prototype
each call
        return builder.build();
    }
}
```

### Lazy initialization

- **@Lazy** delays creation until first use, which speeds up startup but moves wiring errors to runtime. Spring Boot can make everything lazy with spring.main.lazy-initialization=true.

## 6. Circular Dependencies

**The problem:** bean A needs bean B, and B needs A.

- **Constructor injection** cannot resolve it. Spring throws BeanCurrentlyInCreationException at startup, because neither object can be built first.
- **Setter or field injection** historically worked, using an early reference to the half-built singleton from Spring's three-level cache (singletonObjects, earlySingletonObjects, singletonFactories).
- **Since Spring Boot 2.6, circular references are prohibited by default** (spring.main.allow-circular-references=false). Turning that on is a workaround, not a fix.

### Better fixes

- **Redesign:** a cycle usually means two classes share responsibility. Extract the shared logic into a third class that both depend on.
- **Use events:** publish an ApplicationEvent instead of calling the other service directly.
- **Break the cycle at the injection point:** mark one constructor parameter @Lazy, so Spring injects a lazy proxy and resolves the real bean on first use.

```
@Service
class AService {
    private final BService b;
    AService(@Lazy BService b) { this.b = b; }   // breaks the cycle
}

@Service
class BService {
    private final AService a;
    BService(AService a) { this.a = a; }
}
```

# 7. DSA: Binary Search (3 Problems)

### Template to memorize

- Use **mid = lo + (hi - lo) / 2** to avoid integer overflow.
- Decide on the loop form first. **lo <= hi** with lo = mid + 1 and hi = mid - 1 searches for an exact value. **lo < hi** with hi = mid searches for a boundary.
- Always ask: what is the invariant, and does the range shrink every iteration?

## Problem 1: Binary Search

- **Idea:** compare the middle with the target and discard the half that cannot contain it.
- **Complexity:** O(log n) time, O(1) space.
- **Edge cases:** empty array, one element, target smaller than the first or larger than the last element.

```
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

## Problem 2: Search in Rotated Sorted Array

- **Idea:** after splitting at mid, at least one half is sorted. Find the sorted half by comparing nums[lo] with nums[mid]. If the target lies inside the sorted half's range, search it, otherwise search the other half.
- **Complexity:** O(log n) time, O(1) space.
- **Edge cases:** array not rotated, two elements (use nums[lo] <= nums[mid]), target absent. This version assumes no duplicates.

```
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[lo] <= nums[mid]) {                      // left half sorted
            if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                          // right half
sorted
            if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

## Problem 3: Find Minimum in Rotated Sorted Array

- **Idea:** compare nums[mid] with nums[hi]. If nums[mid] > nums[hi], the minimum is to the right of mid. Otherwise it is at mid or to its left.
- **Complexity:** O(log n) time, O(1) space.
- **Edge cases:** array not rotated (answer is the first element), single element.

- **Follow-up to prepare:** with duplicates (Find Minimum II), when nums[mid] equals nums[hi] you can only safely do hi--, so the worst case becomes O(n).

```
public int findMin(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] > nums[hi]) lo = mid + 1;
        else hi = mid;
    }
    return nums[lo];
}
```

## 8. Interview Questions to Answer Aloud

1. What is IoC, and how is it different from DI?
2. BeanFactory vs ApplicationContext: what extra does ApplicationContext give you?
3. Why is constructor injection preferred over field injection?
4. If two beans match a type, how does Spring decide which one to inject?
5. @Component vs @Bean: when do you use each?
6. What does @SpringBootApplication include, and where does component scanning start?
7. Walk through the lifecycle of a singleton bean from creation to destruction.
8. Where in the lifecycle are AOP proxies created?
9. What are the bean scopes? What goes wrong when a prototype bean is injected into a singleton?
10. Is a Spring singleton the same as the Singleton design pattern? Is it thread-safe?
11. What is a circular dependency, and how does Spring handle it for constructor vs setter injection?
12. How would you fix a circular dependency between two services?
13. What is the difference between @PostConstruct and InitializingBean?

## 9. End-of-Day Checklist

- [ ] I can explain IoC and DI, and name the three injection styles with their trade-offs.
- [ ] I can describe the bean lifecycle in order, including where proxies are created.
- [ ] I can explain the prototype-in-singleton problem and two fixes.
- [ ] I can explain why circular dependencies fail and three ways to fix them.

- [ ] I solved all 3 binary search problems and can write the template from memory.
- [ ] I tried the lifecycle and circular-dependency examples in a real Spring Boot project.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

**Tomorrow (Week 2, Day 2):** AOP and proxies (JDK vs CGLIB), Spring Boot auto-configuration and starters, conditional beans, plus stack problems.